# The Shell

Tzu-You Lin

April 2, 2025

# Outline

Introduction

Shell Basics

File Processing

Programming

# Introduction

# What is the Shell?

- A command-line interface for interacting with the operating system
- Allows users to execute commands by typing them
- Provides a scripting environment for automating tasks
- Acts as an intermediary between the user and the system's kernel

# Shell Interface Examples

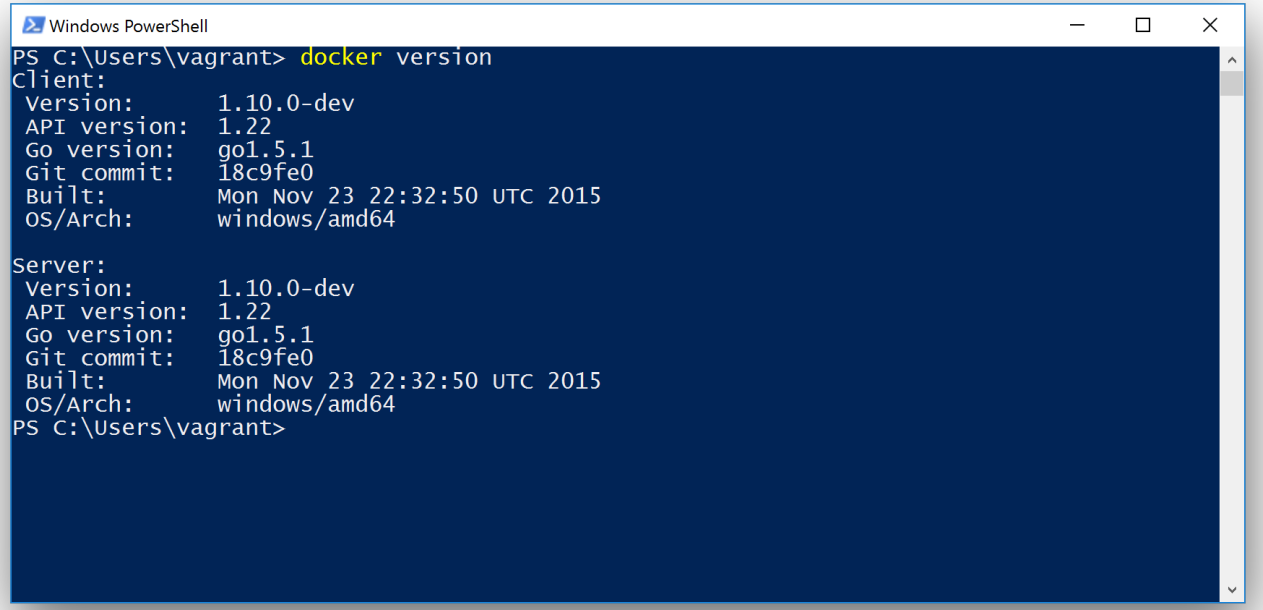


Figure: Windows PowerShell

# Shell Interface Examples (Cont.)

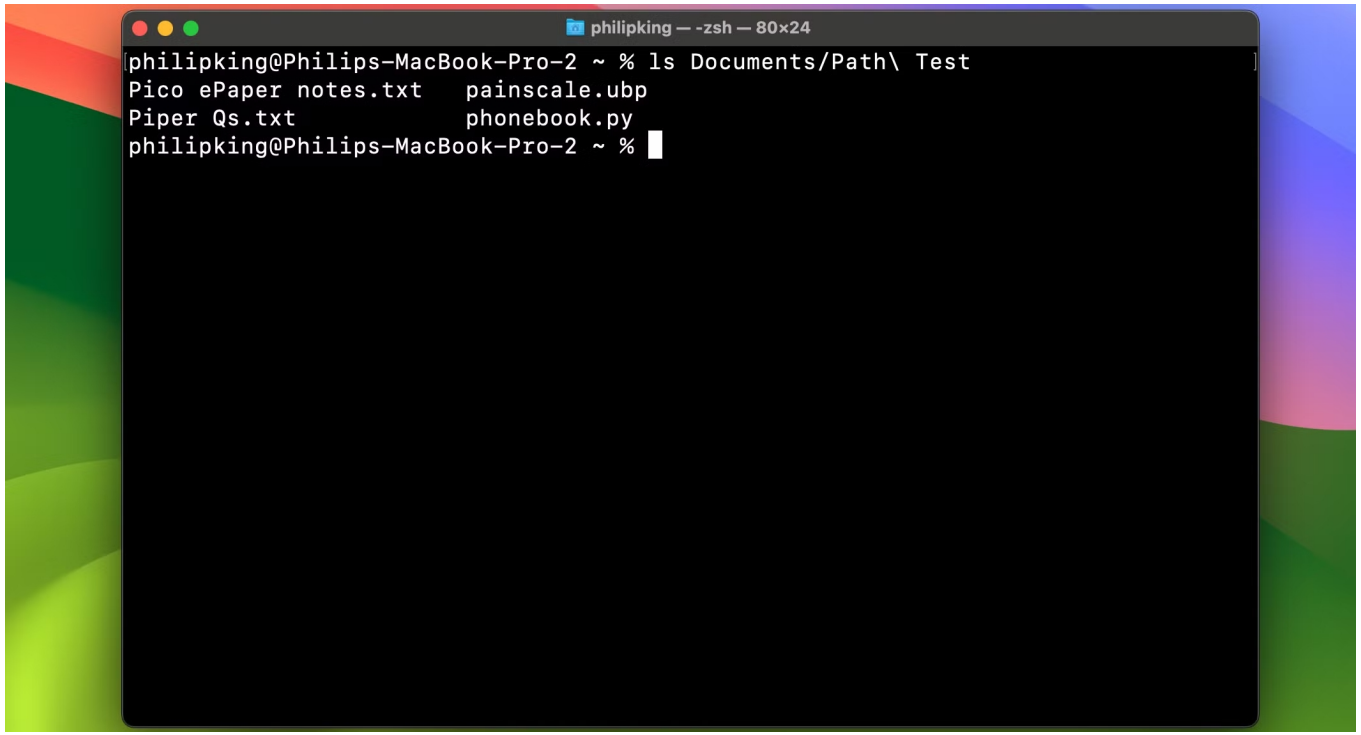


Figure: MacOS Terminal

# Shell Interface Examples (Cont.)

```
mark@linux-desktop: /tmp/tutorial
File  Edit  View  Search  Terminal  Help
Setting up tree (1.7.0-5) ...
Processing triggers for man-db (2.8.3-2) ...
mark@linux-desktop:/tmp/tutorial$ tree
.
├── another
├── combined.txt
├── dir1
├── dir2
│   ├── dir3
│   ├── test_1.txt
│   ├── test_2.txt
│   └── test_3.txt
├── dir4
│   └── dir5
│       └── dir6
├── folder
└── output.txt

8 directories, 5 files
mark@linux-desktop:/tmp/tutorial$
```

Figure: Linux Terminal

# Why Using The Shell?

- Powerful command execution and problem-solving capabilities beyond what GUIs can offer
- Avoids memory complications associated with heavy applications and IDEs
- Efficient text-based interface for system interaction and file manipulation
- Automation and reproducibility through scripting, unlike manual GUI clicking
- Essential for high-performance computing and server administration
- Streamlines complex workflows by combining multiple programs and scripts (e.g., using Makefiles)
- Better visibility and control of system processes

# The Unix Philosophy

- The shell tools we use today originate from the Unix family of operating systems
- Developed at Bell Labs in the 1970s
- Embodies the core Unix philosophy:
  *"Do One Thing And Do It Well"*
- Key principles:
  - Pair and chain well-designed individual components
  - Build powerful complex systems from simple parts
  - Minimalist and modular approach
- This philosophy is clearly reflected in Unix shell design and functionality

# Common Shell Use Cases

- Version Control
  - Managing code repositories with Git
  - Tracking changes and collaborating on projects
  - Branching and merging code
- Development Tasks
  - Compiling source code
  - Running scripts and programs
  - Managing build processes
- File System Navigation
  - Searching for files and directories
  - Organizing and managing files
  - Batch file operations
- System Administration
  - Monitoring system resources
  - Managing processes
  - Installing and updating software

# Learning the Shell: It's Worth It!

- Don't be intimidated by the command line interface
  - Like any tool, it becomes natural with practice
  - Most computer-savvy students adapt quickly
- The learning curve pays off:
  - Increased productivity
  - Better understanding of your system
  - More control over your computing environment
- Personal experience as an instructor:
  - Students who embrace the shell become more confident
  - They often wonder how they worked without it before
  - Many make it their preferred way of working
- Recommendation: Start small, practice regularly, and build up your skills

# Shell Basics

# Terminologies

- The shell goes by many names: Shell, Terminal, TTY, Command Prompt, and Command Line Interface (CLI)
- All refer to the text-based interface for interacting with your computer
- In this course, we'll use Bash (Bourne Again SHell):
  - Default shell on most Linux distributions and macOS
  - Industry standard for shell scripting
  - Rich feature set and extensive documentation
- For Windows users:
  - Install Windows Subsystem for Linux (WSL)
  - Provides a full Linux environment on Windows
  - Enables native Bash usage without virtualization

# Shell Prompt

When you open a shell, you'll see something like:

```
username@hostname:~$
```

- Let's break down each part:
  - `username` Your unique user identifier on the system
  - `@hostname` The name of the computer / server you're using
  - `:~` Your directory path (where `~` signifies the user's home directory)
  - `$` Indicates you're a regular user (vs. `#` for root/admin), and is the start of your command prompt.

# Command Syntax

All Bash commands follow a consistent pattern:

```
command option(s) argument(s)
```

- `command`: The program or action you want to run
- `option(s)`: Modifies how the command behaves (usually starts with - or --)
- `argument(s)`: What the command should act upon (files, directories, text, etc.)

Examples:

```
$ ls -lh ~/Documents/
$ sort -u myfile.txt
```

# Command Syntax (Cont.)

Note:

- Options and arguments are optional:

```
$ ls                    # No options or arguments
$ ls ~/Documents/       # Just an argument
$ ls -lh                # Just options
```

- The command itself is mandatory - you must specify what you want to do

# Getting Help with the man Command

The `man` (manual) command provides detailed documentation for commands:

```
$ man ls     # Shows manual for ls command
```

- Manual pages contain:
  - Command description and purpose
  - Available options
  - Basic examples
  - Common use cases
- Most commands have a manual page - if you're unsure how to use a command, try `man command`

# Root Directory and Home Directory

Two important locations in the filesystem:

- Root Directory (/):
  - The top-level directory in the filesystem
  - All other directories and files are under root
  - Absolute paths always start from root

```
$ ls /
bin    etc    lib    opt    sbin   tmp    var
dev    home   media  proc   sys    usr
```

- Home Directory (~ or /home/username):
  - Personal directory for each user
  - Stores user-specific files and settings
  - Default location when opening terminal

```
$ echo ~          # Shows home directory path
/home/username
$ cd ~            # Goes to home directory
```

# The pwd Command - Print Working Directory

The `pwd` command shows your current location in the filesystem:

```
$ pwd
/home/username/Documents
```

- `pwd` stands for "Print Working Directory"
- Shows the full path from root (/) to your current directory
- Useful when you need to:
  - Confirm your location in the filesystem
  - Get the absolute path of your current directory
  - Debug navigation issues
- No options are typically needed for basic usage

# The cd Command - Change Directory

The `cd` command allows you to navigate between directories:

```
$ cd Documents/     # Move into Documents directory
$ cd ..            # Move up one directory
$ cd ~             # Go to home directory
$ cd /             # Go to root directory
```

- `cd` stands for "Change Directory"
- Common navigation shortcuts:
  - `.` - Current directory
  - `..` - Parent directory
  - `~` - Home directory
  - `/` - Root directory
- You can use absolute paths (from root) or relative paths:

  ```
  $ cd /home/username/Documents    # Absolute path
  $ cd ./projects                  # Relative path
  ```

- If no path is specified, `cd` defaults to home directory

# Handling Spaces in File and Directory Names

**Problem:**
Spaces in filenames create problems in the command line, since the shell treats each word separated by spaces as a different argument:

```
$ cd My Documents     # ERROR: Shell sees two separate arguments
```

**Solutions:**

1. Use quotes around the entire path:

   ```
   $ cd "My Documents"   # Enclose path in quotes
   ```

2. Use escape characters (backslash) or tab completion:

   ```
   $ cd My\ Documents    # Escape the space with backslash
   $ cd My<TAB>         # Tab completion handles spaces
   ```

3. Best practice: Avoid using spaces in directory and file names
   - Use underscores: `My_Documents`
   - Use hyphens: `My-Documents`
   - Use camelCase: `MyDocuments`

# Listing Files and Directories with ls

**Basic Usage:**

- The `ls` command lists files and directories
- Simple usage shows names only:

```
$ ls
Documents  Downloads  Pictures  test.txt
```

**Detailed Information:**

- Use `ls -lh` for detailed listing:

```
$ ls -lh
drwxr-xr-x 2 user group 4.0K Sep 20 10:30 Documents
-rw-r--r-- 1 user group  123B Sep 19 15:45 test.txt
```

**Note:**

- The detailed listing shows permissions, owner, group, size, and timestamps
- For more information about file permissions and other details shown, we encourage you to search online

# Creating Files with touch

**Basic Usage:**

- The `touch` command creates empty files
- Example:

```
$ touch newfile.txt
$ ls
newfile.txt
```

**Note:**

- Creates an empty file if it doesn't exist
- Updates timestamp if file already exists

# Removing Files and Directories with rm

**Basic Usage:**

- The `rm` command removes files
- Example:

```
$ rm oldfile.txt
$ ls
Documents  Downloads  Pictures
```

**Removing Directories:**

- Use `rm -rf` to remove directories and their contents
- Example:

```
$ rm -rf OldDirectory
$ ls
Documents  Pictures
```

**Warning:**

- Be extremely careful with `rm -rf`
- Files and directories are permanently deleted
- No confirmation prompt and no recovery possible
- Double-check what you're deleting before pressing Enter

# Moving Files with mv

**Basic Usage:**

- The `mv` command moves or renames files
- Example of moving:

```
$ mv file.txt Documents/
$ ls Documents/
file.txt
```

- Example of renaming:

```
$ mv oldname.txt newname.txt
$ ls
newname.txt
```

**Note:**

- Will overwrite existing files without warning
- Use `mv -i` for interactive mode with prompts
- Can move multiple files to a directory at once

# Copying Files with cp

**Basic Usage:**

- The `cp` command copies files
- Example of copying a file:

```
$ cp source.txt backup.txt
$ ls
source.txt  backup.txt
```

**Copying Directories:**

- Use `cp -r` for directories
- Example:

```
$ cp -r Documents/ Backup/
$ ls
Documents/  Backup/
```

**Note:**

- Will overwrite existing files without warning
- Use `cp -i` for interactive mode with prompts
- The `-p` flag preserves file permissions and timestamps

# Further Exploration

**For More Advanced File Operations:**

- Learn about the powerful `find` command for searching files
- Explore wildcards (*, ?, [...]) for pattern matching
- These tools will help you work more efficiently with files

**Example Topics to Look Up:**

- Finding files by name, type, or modification time
- Using wildcards with `cp`, `mv`, and other commands
- Combining commands for complex file operations

# File Processing

# Common Data File Formats

**Text-Based Formats:**

- `.csv` (Comma-Separated Values)
  - Simple, readable text format
  - Each line is a record, fields separated by commas
  - Universal support across platforms and tools
- `.txt` (Plain Text)
  - Basic text data, often tab or space-delimited
  - Easy to read and edit manually

**Spreadsheet and Statistical Formats:**

- `.xlsx` / `.xls` (Excel)
  - Microsoft Excel's native format
  - Supports multiple sheets, formulas, and formatting
- `.dta` (Stata)
  - Native format for Stata statistical software
  - Preserves variable labels and metadata
- `.sav` (SPSS)
  - IBM SPSS Statistics format
  - Includes variable and value labels

# Basic Text File Viewing Commands

**Common Commands for Viewing Text Files:**

- `cat` (concatenate)
  - Displays entire contents of file(s)
  - Usage: `cat filename.txt`
  - Can combine multiple files: `cat file1.txt file2.txt`
- `head`
  - Shows first few lines of a file
  - Default: first 10 lines
  - Specify lines: `head -n 5 filename.txt`
- `tail`
  - Shows last few lines of a file
  - Default: last 10 lines
  - Follow updates: `tail -f logfile.txt`
  - Specify lines: `tail -n 20 filename.txt`

# Advanced Text Processing Commands

**Powerful Text Processing Tools:**

- `grep` (Global Regular Expression Print)
  - Searches text patterns in files
  - Usage: `grep "pattern" filename.txt`
  - Supports regular expressions for complex searches
- `sed` (Stream Editor)
  - Performs text transformations on input
  - Common for search and replace operations
  - Example: `sed 's/old/new/g' filename.txt`
- `awk`
  - Powerful text processing programming language
  - Excellent for column-based data processing
  - Can perform calculations and complex text operations

**Note:** These are advanced tools that require significant practice to master.

# Programming

# Running Python Scripts

- Python scripts can be run directly from the shell
- Basic syntax: `python script.py`
- Common variations:
  - Python version specific: `python3 script.py`

# Running R Scripts

- R scripts can be executed from the command line using Rscript
- Basic syntax: `Rscript script.R`
- Common variations:
  - Execute R commands directly: `Rscript -e "print('Hello')"`
- Alternative: Using R interpreter
  - Start R console: `R`
  - Source script: `source("script.R")`

# Understanding Redirection

- Shell redirection allows you to control input/output streams
- Basic redirection operators:
  - `>` - Redirect output to a file (overwrite)
  - `>>` - Append output to a file
  - `<` - Take input from a file
- Examples:
  - Save command output: `ls -l > files.txt`
  - Append to log: `echo "Done" >> log.txt`

# Input/Output Redirection with Scripts

- Scripts can read input and write output to files using redirection
- Example Python script (`count.py`):
  - `for line in sys.stdin:  print(len(line))`
  - Reads each line and outputs its length
- Running with redirection:
  - Input from file: `python count.py < input.txt`
  - Output to file: `python count.py < input.txt > lengths.txt`
  - Append output:
    `python count.py < input.txt >> lengths.txt`

# Automating Testing with Redirection and Diff

- In this course, we encourage you to:
  - Write Python programs that read from stdin and write to stdout
  - Automate testing using input/output redirection
  - Use `diff` to compare expected and actual outputs
- Example testing workflow:
  - Run program:
    ```
    python main.py < test_input.txt > actual_output.txt
    ```
  - Compare outputs:
    ```
    diff expected_output.txt actual_output.txt
    ```
  - No output from diff means outputs match exactly

# References

- These slides are adapted from:
  - Grant McDermott's Data Science for Economists course materials
  - `https://github.com/uo-ec607/lectures`